API Bug Report

Paylocity Benefits Dashboard

STE Assessment — Manual Execution Results

14 bugs found · Postman · May 21, 2026 · Diego Lizarraga

Summary

Environment: https://wmxrwq14uc.execute-api.us-east-1.amazonaws.com/Prod

Tool: Postman

Total bugs found: 14

ID	Title	Severity	Type	Status
BUG-API-001	POST accepts request without required username	P0 — Critical	API Design	Open
BUG-API-002	Client-supplied username always overwritten with auth credentials	P0 — Critical	API Design	Open
BUG-API-003	Salary behavior inconsistent between POST and PUT	P0 — Critical	API Design	Open
BUG-API-003b	Salary updated with PUT contradicts business rules	P0 — Critical	Data Integrity	Open
BUG-API-004	Invalid auth token returns 500 instead of 401	P1 — High	API Design	Open
BUG-API-005	Invalid formatted input in integer field returns 405 instead of 400	P1 — High	API Design	Open
BUG-API-006	GET/DELETE non-existent ID returns 500 with HTML error page	P1 — High	API Design	Open
BUG-API-007	GET/DELETE malformed UUID returns 405 instead of 400 or 404	P1 — High	API Design	Open
BUG-API-008	PUT without ID in body returns 405 instead of 400	P1 — High	API Design	Open
BUG-API-009	PUT with non-existent ID returns 405 instead of 404	P1 — High	API Design	Open
BUG-API-010	DELETE already-deleted employee returns 200 instead of 404	P1 — High	API Design	Open
BUG-API-011	benefitsCost and net returned with 5 decimal places instead of 2	P2 — Medium	Calculation	Open
BUG-API-012	POST without Content-Type header returns 200 instead of 415	P2 — Medium	API Design	Open
BUG-API-013	Swagger only documents 200 responses	P3 — Low	Documentation	Open
BUG-API-014	Swagger defines no response body schema for any endpoint	P3 — Low	Documentation	Open

Bug Details

BUG-API-001

POST accepts request without required username — uses login credentials instead

Severity	**P0 — Critical**
Found in	TC-API-POST-007
Bug Type	Validation Bug + API Design Bug
Status	Open
URL	/Prod/api/Employees

Steps

```
POST /api/Employees  Body: {"firstName":"John","lastName":"Tester","dependants":0}
(username omitted)
```

Expected

400 Bad Request, username is a required field per swagger schema.

Actual

```
200 OK  {"username":"TestUser970","firstName":"John","lastName":"Tester",...}
```

Impact

The API doesn't enforce its own required field rules. Every employee created through the UI has no client-supplied username. The system substitutes the authenticated user's identity, which contradicts the swagger schema.

BUG-API-002

Client-supplied username field always overwritten with authenticated user credentials

Severity	**P0 — Critical**
Found in	TC-API-POST-001, TC-API-POST-017
Bug Type	API Design Bug
Status	Open
URL	/Prod/api/Employees

Steps

```
POST /api/Employees  Body:
{"firstName":"John","lastName":"Tester","username":"johnt2","dependants":0}
```

Expected

Employee created with username: johnt2 as supplied in the request body.

Actual

```
200 OK  {"username":"TestUser970","firstName":"John","lastName":"Tester",...}  // johnt2 ignored
```

Additional Evidence

TC-API-POST-017: A 51-character username was submitted and also overwritten with TestUser970. The API ignores the username field entirely regardless of its value.

Impact

The username field in the request body is non-functional. The swagger schema defines it as a required client-supplied field, the actual behavior contradicts this contract entirely.

BUG-API-003

Salary behavior is inconsistent between POST and PUT

Severity	**P0 — Critical**
Found in	TC-API-PUT-009
Bug Type	API Design Bug
Status	Open
URL	/Prod/api/Employees

Steps

```
POST with salary: 99999  →  ignored, salary stays 52000
PUT  with salary: 99999  →  accepted, gross becomes 3846.12
```

Finding 1

Salary behavior is inconsistent between POST and PUT. POST ignores a submitted salary and keeps the default 52000. PUT accepts and persists the new value. No documentation explains this difference.

Finding 2

When salary is changed via PUT, gross and net are recalculated based on the new salary, contradicting the business rule that all employees earn $2,000 per paycheck.

The real bugs

- Inconsistency between methods → POST ignores salary, PUT accepts it, with no documented reason.
- Salary being replaced with PUT violates the stated business rules (see BUG-API-003b).

BUG-API-003b

Salary being client-settable via PUT violates the business rules

Severity	**P0 — Critical**
Found in	TC-API-PUT-009
Bug Type	Data Integrity Bug
Status	Open
URL	/Prod/api/Employees

Steps

```
PUT /api/Employees
Body:
{"id":"<existing>","firstName":"ReadOnly","lastName":"Test","salary":99999.00}
```

Expected

Business rules state all employees earn $2,000 per paycheck. Salary should be fixed at 52,000 annually. The submitted salary value should be ignored.

Actual

```
200 OK  {"salary":99999,"gross":3846.12,"benefitsCost":38.46,"net":3807.65,...}
```

Impact

Any API consumer can set an arbitrary salary for any employee, causing gross and net to be recalculated based on a fabricated value. In a production payroll application this is a critical data integrity vulnerability — the API directly contradicts the business rule that all employees earn $2,000 per paycheck.

BUG-API-004

Invalid auth token returns 500 Internal Server Error instead of 401

Severity	**P1 — High**
Found in	TC-API-AUTH-002
Bug Type	API Design Bug
Status	Open
URL	All endpoints

Steps

```
GET /api/Employees  Authorization: Basic invalidtoken==
```

Expected

401 Unauthorized → invalid token should be rejected with custom message.

Actual

```
500 Internal Server Error  {"title":"An error occurred while processing your request.","status":500}
```

Impact

The authentication layer crashes on malformed tokens instead of rejecting them. This leavess unhandled exception behavior and could leak server information. The auth layer should validate token format before processing and return 401 for any invalid credential scenario.

BUG-API-005

Malformed input in integer field returns 405 Method Not Allowed instead of 400

Severity	**P1 — High**
Found in	TC-API-POST-018, TC-API-POST-019
Bug Type	API Design Bug
Status	Open
URL	/Prod/api/Employees

Steps

```
POST /api/Employees  Body:
{"firstName":"John","lastName":"Tester","username":"test","dependants":"two"}
```

Expected

400 Bad Request → dependants must be an integer per swagger schema.

Actual

```
405 Method Not Allowed  (empty response body)
```

Also observed for

Float in dependants field (2.5) which also returns 405 with empty body.

Impact

405 Method Not Allowed means the HTTP method isn't supported for this endpoint, which is semantically wrong, POST is clearly supported. The server fails to route the request when it hits unexpected input types and falls into a 405 instead of a proper validation error. Clients can't distinguish a method error from a validation error.

BUG-API-006

GET/DELETE non-existent employee ID returns 500 with full HTML error page

Severity	**P1 — High**
Found in	TC-API-GET-005, TC-API-DEL-002
Bug Type	API Design Bug
Status	Open
URL	/Prod/api/Employees/{non-existent-id}

Steps

```
GET /api/Employees/00000000-0000-0000-0000-000000000000
```

Expected

404 Not Found with a JSON error body.

Actual

```
500 Internal Server Error  Content-Type: text/html
Body: Full ASP.NET Core HTML error page including:
  — "An error occurred while processing your request."
  — "Development Mode" instructions
  — References to ASPNETCORE_ENVIRONMENT configuration
  — Request ID exposed in HTML
```

Impact

Two separate issues: a REST API must never return an HTML response, and the HTML error page exposes internal framework details including ASP.NET Core environment hints and request IDs. A non-existent resource should return a clean JSON 404.

BUG-API-007

GET/DELETE malformed UUID format returns 405 instead of 400 or 404

Severity	**P1 — High**
Found in	TC-API-GET-006, TC-API-DEL-004
Bug Type	API Design Bug
Status	Open
URL	/Prod/api/Employees/not-a-valid-uuid

Steps

```
GET /api/Employees/not-a-valid-uuid
```

Expected

400 Bad Request for invalid UUID format, or 404 Not Found.

Actual

```
405 Method Not Allowed  (empty response body)
```

Impact

Same systemic pattern as BUG-API-005. The server fails to route malformed input and returns 405 instead of a meaningful error.

BUG-API-008

PUT without ID in body returns 405 Method Not Allowed instead of 400

Severity	**P1 — High**
Found in	TC-API-PUT-010
Bug Type	API Design Bug
Status	Open
URL	/Prod/api/Employees

Steps

```
PUT /api/Employees  Body:
{"firstName":"NoId","lastName":"Test","username":"noidtest"}
```

Expected

400 Bad Request — id field is required for update operations.

Actual

```
405 Method Not Allowed  (empty response body)
```

Impact

Part of the systemic 405 pattern. A PUT without an ID is a common mistake — the API should tell the client what's missing, not return an HTTP method error.

BUG-API-009

PUT with non-existent employee ID returns 405 instead of 404

Severity	**P1 — High**
Found in	TC-API-PUT-011
Bug Type	API Design Bug
Status	Open
URL	/Prod/api/Employees

Steps

```
PUT /api/Employees  Body: {"id":"00000000-0000-0000-0000-
000000000000","firstName":"Ghost","lastName":"User"}
```

Expected

404 Not Found, employee with that ID does not exist.

Actual

```
405 Method Not Allowed  (empty response body)
```

Impact

Part of the systemic 405 pattern. A client trying to update a non-existent employee gets a misleading method error instead of a not-found response.

BUG-API-010

DELETE already deleted employee returns 200 OK instead of 404

Severity	**P1 — High**
Found in	TC-API-DEL-003
Bug Type	API Design Bug
Status	Open
URL	/Prod/api/Employees/{id}

Steps to Reproduce

- Create an employee and note the id.
- DELETE /api/Employees/{id} — first call returns 200. Correct.
- DELETE /api/Employees/{id} again with the same id.

Expected

404 Not Found — employee was already deleted.

Actual

```
200 OK  (empty response body)
```

Impact

The API returns success for an operation that had no effect. The server isn't verifying whether the resource exists before confirming deletion. A client receiving 200 has no way to know the resource was already gone. Idempotent DELETE should return 404 on the second call.

BUG-API-011

benefitsCost and net returned with 5 decimal places instead of 2

Severity	**P2 — Medium**
Found in	TC-API-POST-002
Bug Type	Calculation Bug
Status	Open
URL	/Prod/api/Employees

Observed

```
{"benefitsCost":38.46154,"net":1961.5385,...}
Formula:   1000 / 26 = 38.461538...
Returned:  38.46154  (5 decimal places)
Expected:  38.46     (2 decimal places)
```

Impact

Payroll values should be expressed in cents — 2 decimal places. The API returns raw floating point division results without rounding. The UI masks this by truncating the display, but the underlying stored data has incorrect precision. Any downstream system consuming the API receives unrounded monetary values.

BUG-API-012

POST without Content-Type header returns 200 instead of 415

Severity	**P2 — Medium**
Found in	TC-API-POST-030
Bug Type	API Design Bug
Status	Open
URL	/Prod/api/Employees

Steps

```
POST /api/Employees  Headers: Authorization: Basic <token>  (Content-Type omitted)
```

Expected

415 Unsupported Media Type → Content-Type is required for POST requests with a body.

Actual

```
200 OK  Employee created successfully
```

Impact

The API accepts requests without a declared content format. While not a functional bug, it's poor API design, servers should require clients to declare the format of their payload rather than guess it.

BUG-API-013

Swagger only documents 200 responses leaving 400, 401, 404 not defined

Severity	**P3 — Low**
Found in	TC-API-SCHEMA-002
Bug Type	Documentation Bug
Status	Open
URL	swagger/v1/swagger.json

Expected

All endpoints document 200, 400, 401, and 404 responses with schema definitions.

Actual

```
Every endpoint: "responses": {"200": {"description": "Success"}}  // No 400, 401, or 404 documented
```

Impact

Clients have no documented contract for error handling. This assessment documents the actual behavior to fill the gap.

BUG-API-014

Swagger defines no response body schema for any endpoint

Severity	**P3 — Low**
Found in	TC-API-SCHEMA-002
Bug Type	Documentation Bug
Status	Open
URL	swagger/v1/swagger.json

Actual Response Schema (documented here as the missing contract)

```
{"id":"string (UUID)",
 "firstName":"string",
 "lastName":"string",
 "username":"string (always authenticated user — see BUG-API-002)",
 "dependants":"integer",
 "salary":"float (client-settable via PUT — see BUG-API-003)",
 "gross":"float (unrounded — see BUG-API-011)",
 "benefitsCost":"float (unrounded — see BUG-API-011)",
 "net":"float (unrounded — see BUG-API-011)",
 "partitionKey":"string (authenticated username)",
 "sortKey":"string (UUID, same as id)"}
```